AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents orchestrate multi-step activities across users, tools, and system resources. To improve agent quality, safety, and cost efficiency, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Traditional systems profiling answers analogous questions by aggregating resource use over code paths rather than debugging individual executions, but existing agent observability tools do not combine source-linked additive system effects with selectable profiler projections. Agent responsibility lies in semantic categories such as task intent and workflow phase rather than code paths, but trajectories lack stable identifiers and function-call hierarchies for automatic profiling attribution. Agent observability needs profiling, not only debugging. Our *semantic operation stack model* adapts profiling to agent trajectories by representing agent activities as uniform *operations* and replacing runtime stacks with *operation stacks* that attribute the same data hierarchically. AgentProf implements this model as a profiler with pluggable algorithms for *intent attribution* and *stack construction*, compiling local agent trajectories into pprof-compatible profiles. On a fixed suite of 20 real Codex tasks, scoped source lineage reaches 100.0% precision and 96.6% recall, rejects all 1,629 concurrent-control effects, and AgentProf preserves all 1,520 attributed effects during folding. Across real and public trajectories, semantic profiles separate resource use by responsible semantic category and concentrate annotated problems. Out-of-fold evaluation of a pre-specified supervised boundary predictor on 287 session-held-out OSWorld-Human task instances yields 0.739 F1 versus 0.645 for the strongest simple control and 0.816 operation-weighted B^3 partition F1 versus 0.678. AgentProf processes 27,765 operations in 1.17 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, and tool invocations) and a low-level system-effects layer (process spawns and file and network I/O). As agents evolve from short scripts into complex systems, production teams accumulate many trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? At scale, manual trajectory inspection is slow and expensive (Lù et al. 2025), while per-trajectory LLM judging requires a separate evaluator pass for every run (Mazaheri and Mazaheri 2026; Zheng et al. 2023). Profiling provides the aggregation method these questions require. Traditional systems profilers attribute resource consumption to responsible entities rather than debug or trace individual executions.

Existing agent tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Datadog 2025; Laminar 2025). Some now derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026). They do not provide source-linked, additive cross-layer effects as selectable pprof-compatible operation-stack projections.

Adapting profiling to agent trajectories creates two challenges, detailed in Background and Motivation. Traditional profilers use stable function names to identify responsible code paths and runtime call nesting to provide the attribution hierarchy (Google 2024b; Gregg 2011). Agent responsibility instead lies in semantic categories such as task intent and tool operation. Agent trajectories lack both properties. Natural-language prompts provide no stable identifiers for shared intent. Agent events also do not nest like function calls. A prompt sequence may represent one or multiple tasks, and one task may belong to a larger task.

Agent observability needs profiling, not only debugging. Despite these differences, the fundamental profiling methodology transfers to agent trajectories by projecting resources onto responsible entities, assigning stable tags, and attributing resources hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* provide hierarchical attribution, replacing the runtime call stack. Different field choices change the attribution granularity. The same operations can be attributed by task category, execution phase, session, or action type

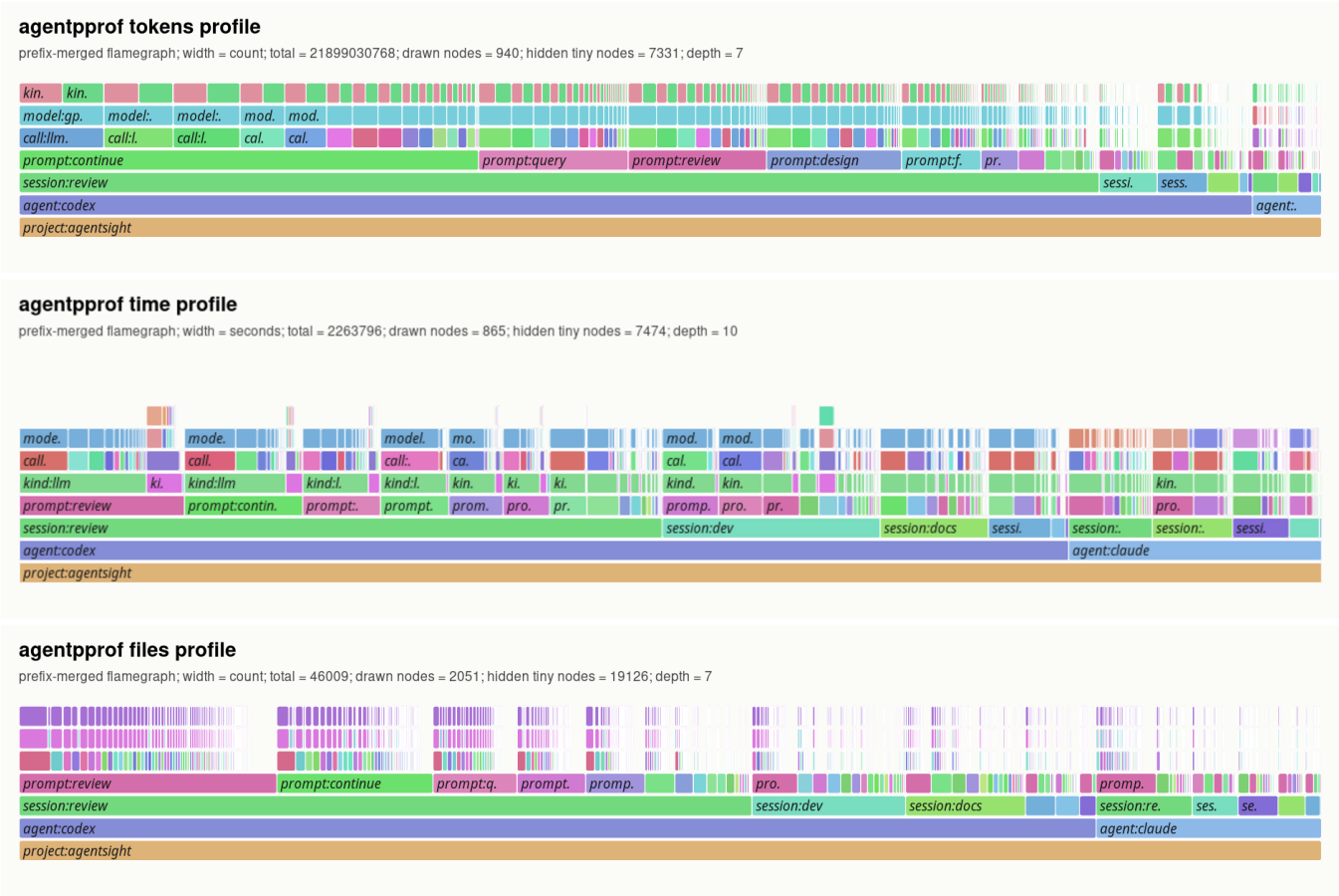

Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. Width represents token count in the top view, wall-clock duration in the middle view, and file-operation count in the bottom view. All three are produced from the same operations by changing only the stack fields and weight function.

(Figure 1).

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Google 2024b) (Figure 2). Two pluggable mechanisms instantiate the model. *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This gives agents the stable tags that traditional profiling gets for free from function names. *Stack construction* builds the attribution hierarchy, either from a user-specified field list or by automatically detecting boundaries between distinct work phases. On a fixed suite of 20 real Codex tasks, the existing scoped source-lineage path reaches 100.0% precision and 96.6% recall while rejecting all 1,629 concurrent-control effects. Current AgentProf then folds all 1,520 attributed effects and exactly preserves the total weight of all five manifest-defined task categories. On 325 real Codex and Claude trajectories (183,714 system observations), prompt tags reduce mixed system-effect weight from 90.4% to 36.7%, while a session-only view leaves 84.4% mixed. Across three complete public benchmarks totaling 27,346 labeled steps (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026a), target-blind semantic profiles improve problem concentration over raw action. Average precision (AP) rises from 0.556 to 0.588 on AgentProcessBench, while inspection work falls from 46.29% to 41.57% at 80% recall on HINTBench and from 46.64% to 19.55% at 50% recall on TraceElephant. On 287 session-held-out OSWorld-Human task instances (Abhyankar, Qi, and Zhang 2025), a pre-specified supervised boundary predictor evaluated out of fold reaches 0.739 F1 versus 0.645 for the strongest simple control and preserves human groups at 0.816 operation-weighted B^3 partition F1 versus 0.678. On the exact union of four public operation workloads, AgentProf builds a 27,765-operation semantic profile in 1.17 s with 464.5 MiB maximum resident set size (RSS), adding 18.2% time and 1.3% memory over a raw-action cost control.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks, developed in Background and Motivation and Design.
2. **System: AgentProf.** A profiler that implements this

model with pluggable intent attribution and stack construction, producing pprof-compatible output, as detailed in Implementation.

3. **Evaluation.** We evaluate source lineage on 20 real Codex tasks, resource views on 325 real trajectories and 15 mapped public families, problem correspondence on three complete public benchmarks, boundary accuracy on 287 held-out task instances, and construction cost on 27,765 operations.

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools for code execution, web search, and file editing. Each tool invocation triggers lower-layer system effects, including process spawns, file reads and writes, and network requests (Zheng et al. 2025). A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

System Profiling

Profiling aggregates measurements to attribute resource consumption to responsible entities, in contrast to single-execution debugging and tracing. In traditional software, the profiling pipeline uses three steps: (1) a profiler periodically samples execution state, (2) it attaches each sample to the current call stack, and (3) it merges samples with identical stacks (Google 2024b; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize aggregate cost as width or node size. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools commonly begin with stable code identifiers and runtime stacks. Pprof can also aggregate labels and promote `tagroot`/`tagleaf` values to pseudo-frames. Agent trajectories instead must first obtain stable semantic responsibility fields and connect them to downstream effects before such profile views exist.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees that support single-run debugging. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing standards (OpenTelemetry 2024; Arize AI 2024b) represent application-supplied attributes in spans, but natural-language histories do not automatically provide stable responsibility categories because prompts or tool invocations expressing the same intent may use different strings.

The second challenge is that agent trajectories have no runtime call hierarchy that automatically supplies semantic attribution. In CPU profiling, the call stack determines attribution at every level. For example, `malloc`'s cost belongs simultaneously to `parse`, `process`, and `main`. A prompt sequence may constitute one task or several. A task may also belong to a larger workflow, yet no runtime mechanism marks these boundaries or determines the appropriate attribution granularity. We quantify this in RQ1 in Evaluation.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

D1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span an intent layer of prompts and LLM calls and a system-effects layer of file I/O and process spawns. The profiler must connect system-layer resource consumption to intent-layer semantic categories.

D2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

D3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy. Function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent span trees do not encode semantic responsibility through function-call nesting, so the profiler must construct attribution hierarchies from the data.

Design

Cross-layer resource projection, stable tags, and hierarchical attribution drive the design. Operations, defined in the Semantic Operation Stack Model, satisfy D1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Intent attribution*, described under Algorithms, satisfies D2 by deriving stable tags from natural-language fields. *Operation stacks* satisfy D3. They replace the runtime call stack with query-time field projections that attribute resources at multiple granularities, allowing the same data to support multiple views without rebuilding the input.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in Background and Motivation, a pro-

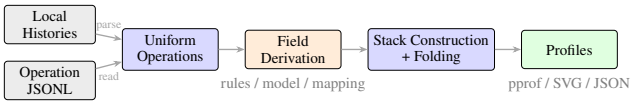

Figure 2: Data-flow and algorithm pipeline. Local histories enter through `agent-session` parsing, while public datasets enter as operation JSONL. Both yield uniform operations before field derivation, stack construction, folding, and profile export.

filer should not distinguish between them in the data representation. AgentProf therefore represents every activity as an `operation`, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values. When a tool invocation triggers system effects, ingestion propagates its semantic fields to the resulting operations, allowing downstream resource weights to fold under the responsible intent.

An `operation stack` provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role. An ordered list of fields $[f_1, f_2, \ldots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore support a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory organized by the dataset's own annotations. Changing the fields changes attribution without changing the data. Sessions and spans are optional fields, not hierarchy levels. The same data supports debugging views with `session` and aggregate profiling views without that field. A view is a triple (φ, σ, w). The predicate φ selects participating operations, the stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and the weight function w assigns a nonnegative measure. Four built-in views cover common questions: (1) `tokens` weights LLM calls by token count, (2) `time` weights timed operations by duration, (3) `files` weights file operations by event count, and (4) `network` weights network operations by event count.

Algorithms

Intent attribution. Intent attribution maps natural-language prompts to short, stable, repeatable tags, giving agent trajectories the stable identifiers that traditional profiling gets for free from function names, as discussed in Background and Motivation. The framework supports three pluggable backends. Explicit rules provide repeatable user control, a local model tags natural-language fields, and unsupervised grouping helps discover candidate categories for later rule authoring. Structured trajectories use the same interface to derive fields from existing action and quality attributes. All backends emit ordinary operation fields, keeping the stack model independent of how a tag was obtained.

Stack construction. Stack construction builds attribution hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When no field list is supplied, AgentProf induces operation stacks automatically. It detects boundaries from visible differences between adjacent operations and uses them to form variable-depth groups, while a configurable depth cap controls stack granularity. This mode produces coarser groups than hand-specified stacks but requires no domain knowledge, making it useful for initial exploration, as evaluated in RQ1.

Implementation

AgentProf implements the semantic operation stack model as an offline Rust CLI (∼9.8K LOC, Figure 2).

Input reconstruction. AgentProf reads Codex and Claude Code local JSONL histories and operation JSONL. AgentSight (Zheng et al. 2025) recordings first pass through a source-specific adapter that emits supported operations; the current CLI does not read those recordings directly. It then reconstructs prompts, LLM calls, tool invocations, and their linked file and network effects.

Field derivation and boundaries. Regex rules are the production default. Each rule has the form `KIND:TAG=REGEX`, and rules use first-match semantics. Agents can refine the rules through repeated AgentProf runs while inspecting unmatched operations. A local LLM tagger, such as a quantized 3B-parameter model through llama.cpp (Gerganov and contributors 2026), generates a single-word tag per prompt with grammar-constrained decoding and caching. TF-IDF (Salton and Buckley 1988) and K-Means (MacQueen 1967) discover candidate prompt groups without predefined rules, while mapping rules derive fields for structured trajectories, for example `phase:navigate=type:(click|scroll|key)`.

Boundary construction. Under a configurable depth bound, the Rust inducer recursively splits adjacent visible-field changes using mean resource-weighted normalized information gain. It accepts the best cut only above $\ln(n)/(2n)$, emits dominant child values as frames, uses query overlap only for exact ties, and excludes scoring or noisy fields.

Profile export. Output formats are pprof protobuf for `go tool pprof -http`, folded stacks for `flamegraph.pl`, standalone SVG flame graphs, and JSON.

Evaluation

Research questions. The evaluation answers four fixed paper-level questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Each following subsection states its protocol, reports the corresponding evidence, and gives the strongest answer supported by that evidence.

We use three classes of data. The real-agent dataset comprises 325 readable trajectories (198 Codex, 50 Claude, and 77 Claude subagent) and 183,714 system observations from a multi-month development project. The public annotated dataset comprises 15 real agent or human execution families converted by mapping rules into 47,590 operations. These cover web agents (WebLINX (Lù, Kasner, and Reddy 2024), Mind2Web (Deng et al. 2023), AgentTrek (Xu et al. 2025), WebShop (Yao et al. 2022)), API agents (API-Bank (Li et al. 2023), ToolBench (Qin et al. 2024), tau-bench (Yao et al. 2025)), coding agents (SWE-agent (Yang et al. 2024)), and mobile/GUI agents (AndroidCtrl (Li et al. 2024), GUI-Odyssey (Lu et al. 2025), ScaleCUA (Liu et al. 2026b)). Four mapped families provide structured quality or boundary annotations, including AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026b), AgentNet (Wang et al. 2025), and OSWorld-Human (Abhyankar, Qi, and Zhang 2025). AgentProcessBench, HINTBench, and TraceElephant form the public problem-localization set and provide the independent targets for RQ2 (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026a). Of HINTBench's reported 629 trajectories, the current official test snapshot enumerates 536. We use all 536 after selecting the field order on the separate 80-trajectory validation snapshot (Wang et al. 2026b).

RQ1: Resource Attribution

RQ1 asks whether semantic profiling improves resource attribution beyond what traditional tag-free tools provide. We first test whether a source-linked path assigns system effects to a declared agent process/tool lineage while rejecting concurrent controls. The fixed suite runs 20 real Codex tasks and one concurrent negative control per task through the existing R114-compatible AgentSight 0.2.37 capture path, then converts the scoped joined effects to operation JSONL and folds them once with the current `agentpprof 0.2.37` binary.

All 20 tasks and controls complete. The scoped join recovers 1,520 of 1,574 in-scope effects with no false positive (100.0% precision, 96.569% recall), and none of 1,629 control effects joins. AgentProf emits 1,520 samples in 152 stacks, exactly preserving the input total and all five manifest-category weights.

This result establishes source fidelity for the suite's declared process/tool scope and lossless folding by AgentProf. The task categories come from the fixed manifest rather than automatic task inference. We run a semantic-axis ablation on 325 real Codex/Claude trajectories (183,714 system observations). The prompt tags were produced by the local 3B-model backend described in Implementation and are treated here as declared categories, not as an independent correctness oracle. We progressively add four tag-axis configurations: (1) no tags, (2) session only, (3) prompt only, and (4) both. We then measure mixed-weight percentage, defined as the fraction of system-effect cost in groups containing observations from multiple task categories. A lower mixed-weight percentage means fewer effects from different prompt-tag categories remain in the same responsibility group. The residual percentage measures weight in groups where the majority category accounts for less than 90% of the group, capturing partial mixing.

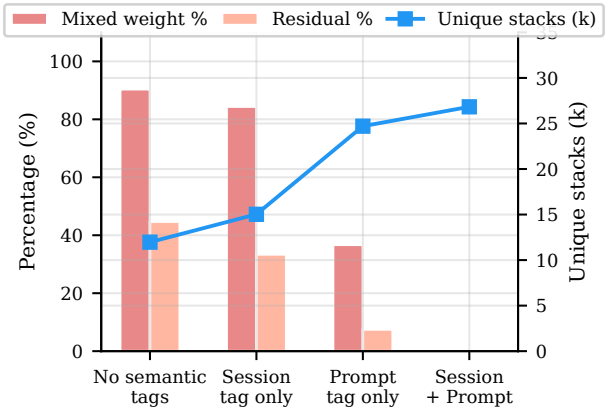

Figure 3: RQ1 semantic-axis ablation on 183,714 system observations from 325 real trajectories. Bars show mixed weight and residual percentages on the left axis, and the line shows unique stacks on the right axis. Adding prompt tags reduces mixed weight from 90.4% to 36.7% while increasing unique stacks from 12k to 25k. Session tags alone barely help (84.4%).

Figure 3 shows the result. Without semantic tags, nearly all system-effect cost falls into mixed groups. For example, a flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. The prompt tag is the decisive axis because it reduces both mixed weight and residual by more than half and nearly doubles the number of unique stacks. It separates observations that share the same system call but originate from different task intents. The session tag alone barely helps because trajectories typically contain multiple intent phases.

A tag-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. The RQ1 experiment measures attribution conditional on the declared prompt tags. The independent RQ3 experiment evaluates group-boundary accuracy. These results show that prompt-level semantic fields materially refine responsibility views beyond tag-free aggregation.

Beyond tag separation, five field selections fold the same 13,265 annotated operations into 9 dataset, 57 phase, 226 semantic, 455 action, or 3,757 per-session groups. Unlike a single-tag GROUP BY, these depths answer task-budget, action-duplication, and session-detail questions. The same model covers all 15 trajectory families without type-specific objects.

The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$), and `network`-tagged prompts rank 8th by time but 93rd by tokens. Thus, multiple weights expose bottlenecks that one metric would miss.

Without user-specified fields, automatic induction on six annotation tasks produces a median of 12 groups, compared with 285 for the per-session view and one for the flat view, and yields variable-depth stacks on 4/6 tasks. We use independent

Table 1: Existing RQ2 controls. Lower Work is better; † is descriptive. AP and Work are not averaged.

Metric	AgentProf	Raw	Other existing views
AgentProcess AP	.588	.556	flat .325; session .599; step .777
HINT Work@80	.416	.463	native .579; session .591; step 1.00
Trace Work@80	1.00	.719	width .677; native/session 1.00
Trace Work@50†	.195	.466	width .396; session .568; native .578

problem annotations to score each group's problem density. AP measures ranking quality, and inspection work measures the operations traversed in that ranking. Automatic induction achieves a median AP of 0.276 versus 0.312 for hand-specified stacks, while reducing inspection work to 65.3% of flat summaries without user configuration.

Figure 1 shows the resulting flame graphs for the `tokens` and `files` views, illustrating that changing only the stack fields and weight function produces different aggregates from the same operations. Together, these results answer RQ1. Scoped source lineage rejects concurrent controls, AgentProf preserves attributed weight across runs, and semantic profiling provides multi-resolution, multi-weight, and automatically induced attribution views over the resulting operations.

RQ2: Problem Correspondence

RQ2 asks whether profiler output corresponds to real problems by testing whether a target-blind profile concentrates independently annotated failures or risky steps into groups that a developer can inspect early. We evaluate complete public workloads from AgentProcessBench (Fan et al. 2026), HINTBench (Wang et al. 2026b), and TraceElephant (Chen et al. 2026a). HINTBench development labels select among 24 fixed field orders. Its reported test labels and all AgentProcessBench and TraceElephant targets remain held out from tag, stack, and ranking construction. Only the final scorer reads them to compute inspection work or, for AgentProcessBench, operation-weighted average precision (AP) within each family and its equal-family macro average. AgentProcessBench uses 20 released judge votes as its step signal and ranks groups by mean risk. For HINTBench and TraceElephant, released localization predictions provide the step signal, and groups are ranked by the maximum 95% Wilson lower bound (Wilson 1927) on hit density over their profile prefixes. The raw-action control changes only the grouping field to each workload's released action while keeping the step signal, ranking rule, scorer, input, and full execution path fixed.

AgentProcessBench contains 1,000 BFCL, GAIA, HotpotQA, and τ-bench trajectories with consensus labels for 8,509 assistant steps (Fan et al. 2026). The released target-blind risk signal raises equal-family macro AP from 0.556 to 0.588 (gain 0.0315; 95% interval $[0.0151, 0.0535]$). Shuffling assignments within raw-action groups while preserving subgroup sizes gives $p = 0.00995$, isolating semantic-specific concentration. Its 419 recurring groups lie between raw action's 259 and the 1,000 session or 8,509 per-step views, which reach higher AP and preclude uniform dominance.

On 536 HINTBench test trajectories, AgentProf reaches 80% macro recall at 41.57% work versus 46.29% raw, 57.93% native, 59.14% session, and 100% independent step. Paired intervals exclude zero for the latter three but not raw ($[-0.2937, 0.0086]$), so this supports the complete profile/prefix/scorer pipeline rather than hierarchy alone. On TraceElephant's 220 failures, AgentProf reaches 50% macro decisive-step recall at 19.55% work versus 46.64% raw and, at 20% work, recovers 52.57% versus 23.79%. This descriptive early region ends in a large tied tier that requires full work at prospective 80% recall, where the interval crosses zero.

Together, the complete workloads answer RQ2 positively: AgentProcessBench isolates semantic-specific concentration, HINTBench supports the complete pipeline, and TraceElephant shows strong early localization. A secondary six-task analysis (34,539 task-operation instances) reaches 25% recall at 20.00% work and 16 groups versus fixed-session's 24.95% and 50; flat needs 100% work, while raw reaches 19.93% and 13 groups. Thus operation stacks reduce session fragmentation without claiming universal semantic dominance.

RQ3: Tag Accuracy

RQ3 asks how accurately categorical operation fields recover independent annotations. RQ3 covers task, phase, and action labels produced by intent attribution or mappings, as well as group boundaries used to construct the hierarchy. We evaluate task partitions and group boundaries; phase, action, and literal tag names remain outside the current evidence.

We hypothesize that pre-specified target-blind taggers or mappings recover accurate, stable task, phase, and action labels and group boundaries on unseen agent and task families. Each component is scored against its corresponding independent annotations and then profiled with the original additive weights.

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations, covering 3,978 operations, 3,691 adjacent pairs, and 2,042 groups. For the supervised Bernoulli Naive Bayes predictor (McCallum and Nigam 1998), we fixed the model family, nine input fields, adjacent-pair feature construction, and training procedure before evaluation. Each session-blocked fold fits parameters and a threshold only on training sessions, then predicts every held-out session once. The predicted boundaries define an ordinary group field that AgentProf projects and folds through its released profile-construction path. This test covers supervised boundary prediction and field integration, not the built-in Rust stack inducer described in Implementation. The three simple controls place a boundary after every operation, whenever the visible action field changes, or whenever the visible phase field changes. We report boundary precision, recall, and F1, plus operation-weighted B^3 partition F1 (Bagga and Baldwin 1998), whose per-operation predicted–human group overlap captures merging and fragmentation.

The supervised predictor reaches 0.739 boundary F1, 0.094 above the strongest simple control, and has the highest

Table 2: Session-held-out RQ3 boundary fidelity on OSWorld-Human. B^3 measures agreement between predicted and human operation partitions.

Method	Prec.	Recall	Boundary F1	B^3 F1
Supervised predictor	0.700	0.782	**0.739**	**0.816**
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 3: RQ4 profile-construction cost. Times are three-run medians, and RSS is the largest observed semantic-profile peak.

Workload	Ops	Sem. (s)	Raw (s)	RSS (MiB)
AgentReward	729	0.04	0.03	19.0
Satraj	4,285	0.17	0.14	73.5
OSWorld	6,010	0.25	0.21	108.4
AgentNet	16,741	0.71	0.58	278.4
Union	27,765	1.17	0.99	464.5

boundary F1 in all five held-out folds. Its groups reach 0.816 B^3 F1 against the human partition, compared with 0.678 for the strongest control. Equivalently, $(1 - B^3\ \mathrm{F1})$ falls from 0.322 to 0.184. AgentProf projects all 3,978 operations onto the predicted group field, producing 2,249 stacks while preserving the total weight of all 3,978 operations.

Using the existing target-blind TF-IDF/K-Means backend, we also infer task partitions from label-hidden text. On all nine available Mind2Web sessions (49 operations) and 100 ScienceWorld sessions (2,504 operations) (Deng et al. 2023; Wang et al. 2022), it reaches operation-weighted V-measure 0.557 and 0.815 at full coverage, versus 0 for a constant tag. This label-permutation-invariant metric tests partitions, not literal tag names.

Together, these experiments provide positive RQ3 evidence across three public families: target-blind task partitions and session-held-out operation groups.

RQ4: Profiling Cost

RQ4 asks what it costs to construct profiles after a session ends. Because AgentProf constructs profiles after execution, we measure post-session time and resources, excluding capture. We run `agentpprof 0.2.37` three times on four complete public workloads and their union, comparing a fixed semantic hierarchy with an identical-input raw-action control through the full parse, construct, fold, and serialize path. The measurements run on a 24-core Intel Core Ultra 9 285K machine with 125 GiB RAM and Linux 6.15.11.

Both time curves are monotonic across the five measured input sizes. The semantic curve has a descriptive slope of 0.0422 ms per operation ($R^2 = 0.9997$), reaching 23,731 operations/s on the union. At that largest input, semantic construction adds 180 ms (18.2%) and 6.0 MiB maximum RSS (1.3%) over raw action.

On matched inputs from eight real Codex sessions, the predecessor AgentFlame used 60 local-model calls and 1.64 s; cached reconstruction used no calls and 0.11 s with 76/76 hits. Thus, shared tag caching avoids repeated field derivation; this predecessor result is not timing for the current `agentpprof` binary. Together, RQ4 shows practical, predictable construction over the tested range and that cached derivation avoids repeated model work.

Scope and Limitations

The evaluation covers offline profiling on tested local and public trajectories; it excludes live-agent overhead because AgentProf profiles after execution.

RQ1 covers the fixed 20-task suite, its declared process/tool scope, and the R114-compatible AgentSight 0.2.37 path; manifest categories are fixed inputs, not inferred tasks.

RQ3 establishes task-partition fidelity on Mind2Web and ScienceWorld and session-held-out group-boundary fidelity on OSWorld-Human. Its fixed phase and action components require matched annotations and pre-specified mappings.

Related Work

Agent observability tools combine per-execution traces with tag, metadata, signal, and metric aggregation (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026c; Datadog 2025; Laminar 2025). LangSmith Insights and Datadog Patterns add cross-trace category hierarchies and aggregate metrics (LangChain 2026; Datadog 2026). AgentProf adds source-linked agent/system operations, conserved weights, and selectable pprof-compatible operation-stack projections.

Traditional profilers aggregate stack samples and can incorporate supplied labels into profile views (Google 2024b; Gregg 2011). Trace systems such as Perfetto support flexible event queries (Google 2024a). AgentProf derives these fields from agent histories, connects them to downstream effects, and constructs alternative hierarchies over the same operations.

Recent agent-diagnosis work spans step- or span-level localization (Barke et al. 2026; Wang et al. 2026a; Liu et al. 2026a), validation with root-cause analysis (Mulian et al. 2026), and diagnostic taxonomies and audit protocols (Mazaheri and Mazaheri 2026). AgentProf complements these methods with category-level aggregate profiling across trajectories, exposing recurring responsibility groups rather than only one execution's fault location.

Conclusion

Agent observability needs profiling, not only debugging. AgentProf builds semantic operation stacks with pluggable intent attribution. Across 20 real Codex tasks, lineage reaches 100.0%/96.569% precision/recall, rejects 1,629 controls, and preserves 1,520 effects. Across 325 real trajectories and three public benchmarks, semantic fields separate effects and target-blind stacks concentrate problems; on 287 held-out OSWorld-Human tasks, a pre-specified predictor reaches 0.739 F1 and 0.816 operation-weighted B^3 F1. AgentProf profiles 27,765 operations in 1.17 s, complementing run-local debugging with cross-trajectory semantic profiles.

References

Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024a. Arize Phoenix. https://arize.com/docs/phoenix.

Arize AI. 2024b. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85. Montreal, Quebec, Canada: Association for Computational Linguistics.

Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gerganov, G.; and contributors. 2026. llama.cpp: LLM Inference in C/C++. GitHub repository.

Google. 2024a. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

Google. 2024b. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.

Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.

Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026a. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.

Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.

Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML*, volume 235 of *Proceedings of Machine Learning Research*, 33007–33056. PMLR.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Proceedings of COLM*.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297. University of California Press.

Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48. AAAI Press.

Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in Agentic Systems. In *AGENT@ICSE*.

OpenAI. 2025. OpenAI Codex CLI. GitHub repository.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Salton, G.; and Buckley, C. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing and Management*, 24(5): 513–523.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Association for Computational Linguistics.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026c. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proceedings of ICLR*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.